

AEM Environment Setup

Study Guide for Beginners

Building, running, and testing AEM code on your own machine before sending anything to the cloud.

Volume 4 of the AEM study material series.

Table of contents

Table of contents	3
1. Introduction	5
What this guide covers	5
2. The three components	7
The components.....	7
3. Development tools	8
The four tools	8
Verify each install	8
4. Adobe I/O CLI	9
Installation	9
Plugins	9
Authentication.....	9
5. File system organization.....	11
Recommended structure	11
6. The QuickStart Jar	12
Two important rules.....	12
Default ports.....	12
7. QuickStart Jar naming convention.....	13
Format.....	13
Examples	13
Tier is locked on first startup.....	13
8. Simulating content distribution	15
Steps to enable	15
9. Local Dispatcher	16
Starting the Dispatcher	16
Validator	16
10. Maven project structure.....	18
Immutable — cannot be changed at runtime	18
Mutable — can be changed at runtime	18
11. Maven modules and package types	19
The modules.....	19
Package types.....	19
The all package.....	19
12. Repo Init and the AEM Archetype	21
Repo Init.....	21

AEM Project Archetype21

13. Troubleshooting23

 The five-step diagnostic flow23

 Most common errors.....23

14. Closing.....25

 What you now have25

 Key concepts to remember for the exam25

1. Introduction

Now that we understand what AEM is, how Cloud Manager works, and the code quality standards, it's time to set up a local development environment.

The goal here is to be able to develop, build, and test AEM code on your own machine before sending anything to the cloud.

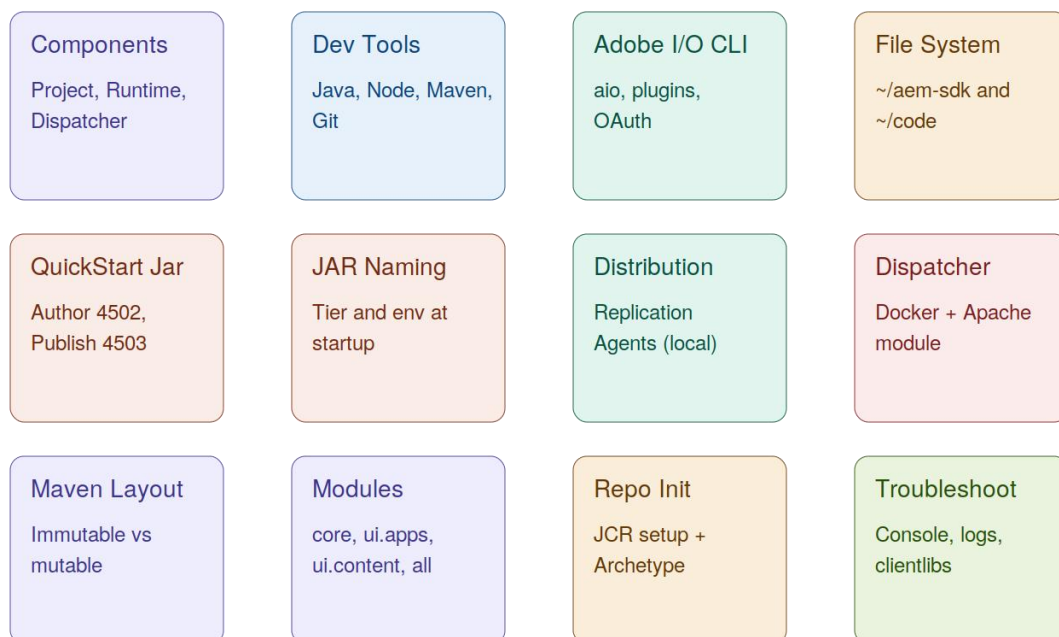
This is one of the most practical lessons in the certification.

What this guide covers

- The three components of a local AEM environment.
- Development tools you must install (Java, Node, Maven, Git).
- The Adobe I/O CLI and OAuth Server-to-Server authentication.
- File system organization and the QuickStart Jar.
- How to simulate content distribution between Author and Publish locally.
- Setting up the local Dispatcher with Docker.
- The Maven project structure and module conventions.
- Repo Init and the AEM Project Archetype.
- A practical troubleshooting checklist.

Setting up a local AEM environment

A roadmap of what this guide covers



Goal: validate everything locally before pushing to Cloud Manager.

Figure 1. A roadmap of every topic in this guide.

2. The three components

A local AEM environment has three logical parts.

The components

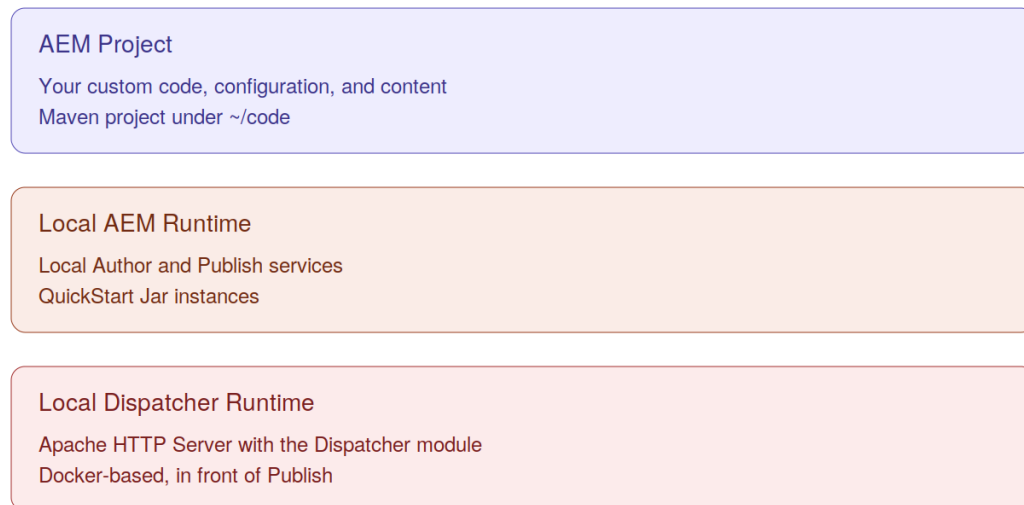
- The AEM Project — your custom code, configuration, and content.
- The Local AEM Runtime — a local version of the Author and Publish services.
- The Local Dispatcher Runtime — a local version of Apache HTTP Server with the Dispatcher module.

Think of it as a miniature version of what runs in the cloud, all on your laptop.

The goal is to validate everything locally before it goes through Cloud Manager.

The three components of a local AEM environment

A miniature version of what runs in the cloud



Goal: develop, build, and test on your laptop before sending anything to the cloud.

Figure 2. The three logical parts of a local AEM environment.

3. Development tools

To work with AEM locally, you need four core tools.

The four tools

- Java JDK 21 — AEM is a Java application, and Java 21 is the recommended version.
- Node.js — required for front-end tooling and the Adobe I/O CLI.
- Apache Maven — the build tool for all AEM projects.
- Git — source control.

Verify each install

After installing each one, always verify with a version command:

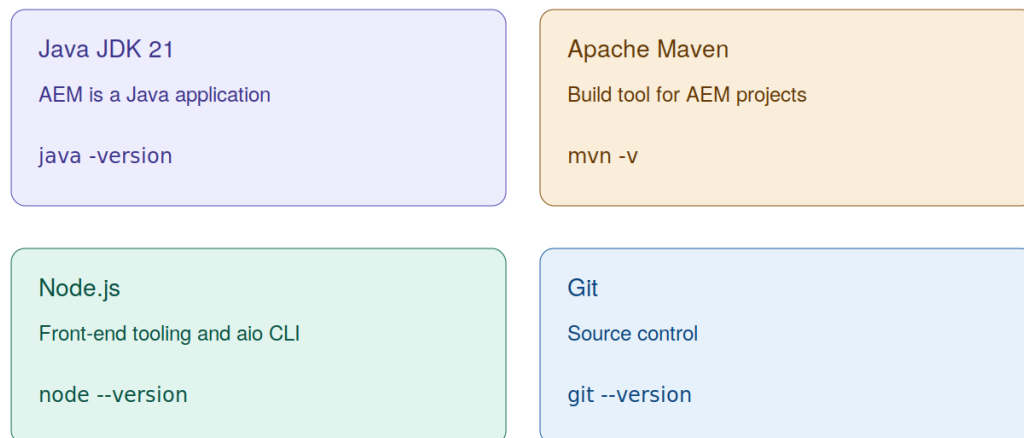
```
java -version
```

```
mvn -v
```

```
node --version
```

The four core development tools

Install → verify with a version command



Java 21 is the recommended version for AEM.

Always verify each install with the version command before moving on.

Figure 3. The four core development tools and their verify commands.

4. Adobe I/O CLI

The Adobe I/O CLI, known as aio, is a command-line tool that lets you interact with Adobe services.

Installation

You install it as a global npm package:

```
npm install -g @adobe/aio-cli
```

Plugins

Once installed, you add plugins for the services you need:

- Cloud Manager plugin — manage pipelines, environments, and deployments.
- AEM Rapid Development Environment (RDE) plugin — push code to a sandbox for fast iteration.
- Asset Compute plugin — develop and test custom asset workers.

Authentication

Important: the old JWT authentication was removed in January 2025.

The only supported method now is OAuth Server-to-Server, configured through the Adobe Developer Console.

Adobe I/O CLI (aio)

Command-line tool to interact with Adobe services

1. Install globally

```
npm install -g @adobe/aio-cli
```

2. Add plugins for the services you need

Cloud Manager plugin

Manage pipelines, environments, deployments

AEM Rapid Development Environment (RDE)

Push code to a sandbox for fast iteration

Asset Compute plugin

Develop and test custom asset workers

3. Authenticate

JWT (deprecated)

Removed January 2025.

No longer supported.

OAuth Server-to-Server

The only supported method now.

Configured in Adobe Developer Console.

Figure 4. Install, plug in, and authenticate the Adobe I/O CLI.

5. File system organization

Adobe recommends a specific folder structure for local development.

Recommended structure

~/aem-sdk/

author/ → local Author service

publish/ → local Publish service

dispatcher/ → local Dispatcher Tools

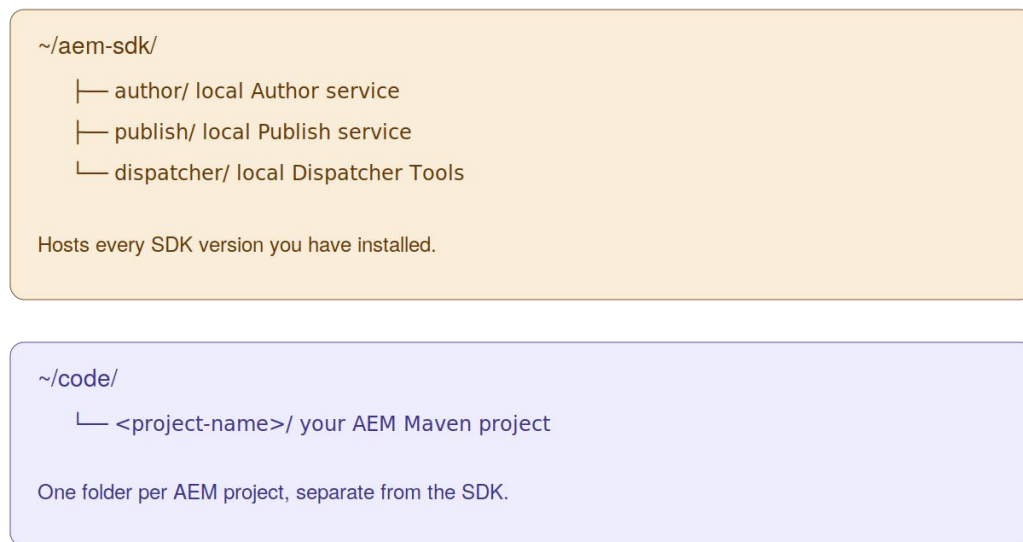
~/code/<project-name>/ → your AEM Maven project

This keeps everything organized and makes it easy to manage multiple SDK versions.

On Windows, the tilde (~) maps to %HOMEPATH%.

Recommended folder structure

Adobe's convention for local AEM development



On Windows, the tilde (~) maps to %HOMEPATH%.

Figure 5. The recommended folder layout for local AEM development.

6. The QuickStart Jar

The AEMaaCS SDK includes a QuickStart Jar, which is a self-contained JAR file that runs a full AEM instance locally.

You download it from Adobe Software Distribution, and you can use it to run either Author or Publish.

Two important rules

- Never double-click the JAR file — it must be started from the command line so the filename conventions for tier, environment, and port apply correctly.
- Always start it from the command line:

```
java -jar aem-author-p4502.jar
```

Default ports

- Author runs on port 4502.
- Publish runs on port 4503.

The QuickStart Jar

Self-contained JAR that runs a full AEM instance locally

Author Authoring environment for content Port 4502 aem-author-p4502.jar	Publish Public-facing render service Port 4503 aem-publish-p4503.jar
---	--

Two important rules

1. Never double-click the JAR file
It must be started from the command line so the filename conventions for tier, environment, and port apply correctly.
2. Always start it like this

```
java -jar aem-author-p4502.jar
```

 From the directory that contains the JAR file.

Source: Adobe Software Distribution (part of the AEMaaCS SDK).

Figure 6. The QuickStart Jar: ports, naming, and the two startup rules.

7. QuickStart Jar naming convention

The filename of the JAR controls how AEM starts up.

Format

aem-<tier>_<environment>-p<port>.jar

Examples

- aem-author-p4502.jar — Author in Dev mode on port 4502.
- aem-publish-p4503.jar — Publish in Dev mode on port 4503.
- aem-author_prod-p4502.jar — Author in Production mode.

Tier is locked on first startup

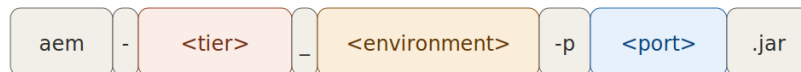
The tier — author or publish — is locked on first startup. If you need to change it, you must delete the crx-quickstart folder and start fresh.

The environment mode is useful for testing environment-specific OSGi configurations.

QuickStart Jar naming convention

The filename controls how AEM starts

Format



Examples

aem-author-p4502.jar
Author in Dev mode on port 4502

aem-publish-p4503.jar
Publish in Dev mode on port 4503

aem-author_prod-p4502.jar
Author in Production mode (env-specific config)

Tier is locked on first startup

Once an AEM instance has started as Author or Publish, the tier cannot be changed. To switch, you must delete the crx-quickstart folder and start fresh with the new JAR name.

The environment mode is useful for testing environment-specific OSGi configurations.

Figure 7. Anatomy of the QuickStart Jar filename.

8. Simulating content distribution

In production, content moves from Author to Publish through the Adobe Pipeline, which is a cloud-only microservice.

Locally, you simulate this using legacy Replication Agents.

Steps to enable

From the Author service:

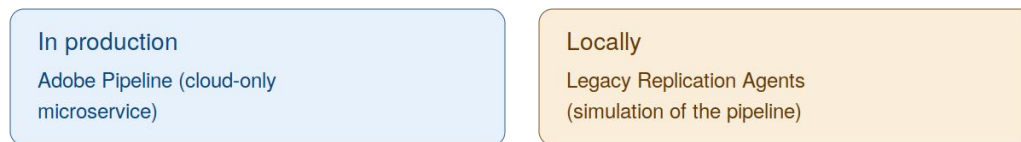
- Open `/etc/replication/agents.author.html`
- Open the Default Agent
- Enable it and point the transport URI to `localhost:4503`

After this, anything you publish on Author will replicate to your local Publish instance.

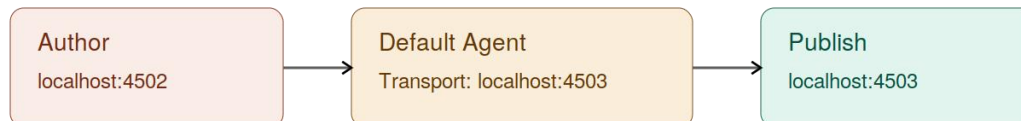
Simulating content distribution locally

Author publishes content to Publish via Replication Agents

Cloud vs local



Local replication flow



Steps to enable

- Open `/etc/replication/agents.author.html` on the Author service
- Open the Default Agent configuration
- Enable it and set the transport URI to `http://localhost:4503`

After this, anything you publish on Author replicates to your local Publish.

Figure 8. Local replication flow: Author → Default Agent → Publish.

9. Local Dispatcher

The Dispatcher Tools are Docker-based. They run an Apache HTTP Server with the Dispatcher module inside a container.

Starting the Dispatcher

You start it with a single script:

```
./bin/docker_run_hot_reload.sh ./src localhost:4503 8080
```

This connects the Dispatcher to your local Publish service and exposes it on port 8080.

The hot-reload variant is preferred because it reloads configuration automatically when files change.

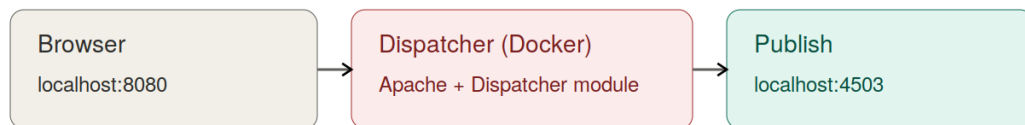
Validator

There is also a validator tool to check your Apache and Dispatcher configuration for errors before you deploy:

```
./bin/validate ./src
```

Local Dispatcher

Docker container running Apache HTTP Server with the Dispatcher module



Start the Dispatcher (hot-reload variant)

```
./bin/docker_run_hot_reload.sh ./src localhost:4503 8080
```

Reloads configuration automatically when files change.

The three arguments

`./src` Path to your Dispatcher configuration

`localhost:4503` Address of your local Publish service

`8080` Port the Dispatcher exposes on the host

Validator tool

```
./bin/validate ./src
```

Figure 9. Browser → Dispatcher (Docker) → Publish, with the run script and arguments.

10. Maven project structure

This is where things get important for the exam.

AEM enforces a clear separation between code and content. The repository has two types of areas.

Immutable — cannot be changed at runtime

- /apps — your custom components, scripts, clientlibs.
- /libs — AEM platform code, never touch this.

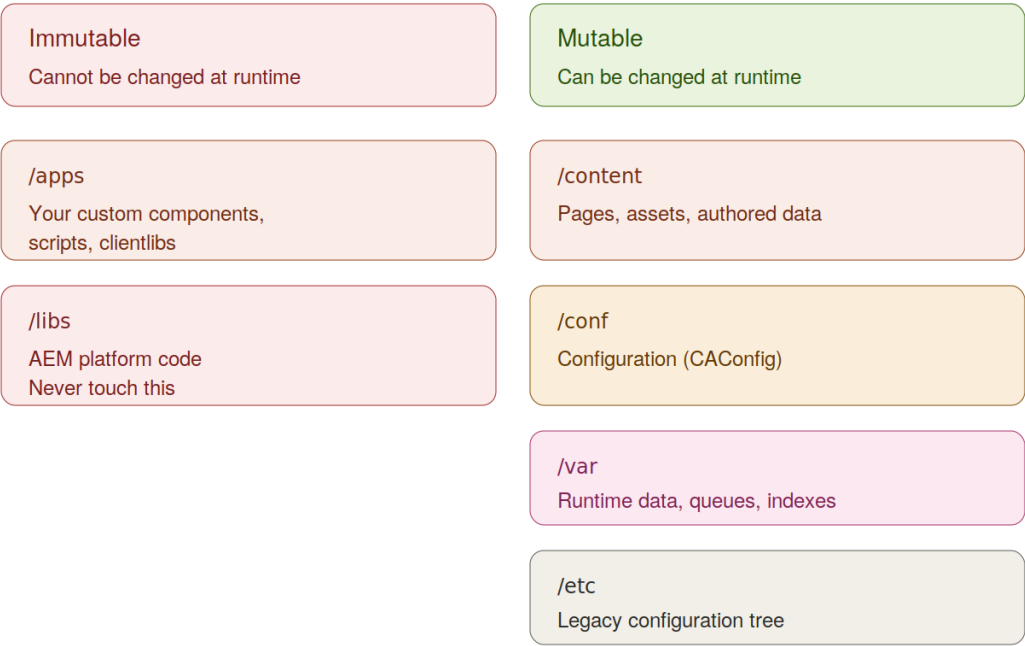
Mutable — can be changed at runtime

- /content — pages, assets, authored data.
- /conf — configuration (CAConfig).
- /var — runtime data, queues, indexes.
- /etc — legacy configuration tree.

Each type of content belongs in a specific Maven module — covered next.

Maven project structure: code vs content

AEM enforces a clear separation between the two



Each type of content belongs in a specific Maven module — covered next.

Figure 10. Immutable vs mutable repository areas.

11. Maven modules and package types

A standard AEM project has these key modules.

The modules

- core — Java OSGi bundle (services, models, servlets).
- ui.apps — code package, deploys to /apps.
- ui.content — content package, deploys to /content and /conf.
- ui.config — OSGi configurations and Repo Init scripts.
- all — the container package, the only artifact deployed by Cloud Manager.

Package types

- application — for code packages like ui.apps.
- content — for content packages like ui.content.
- container — for the all package.

The all package

The all package embeds all other modules and is the single deployment artifact.

All other packages must set cloudManagerTarget to none so Cloud Manager ignores them.

Maven modules and package types

Each module deploys a specific kind of content

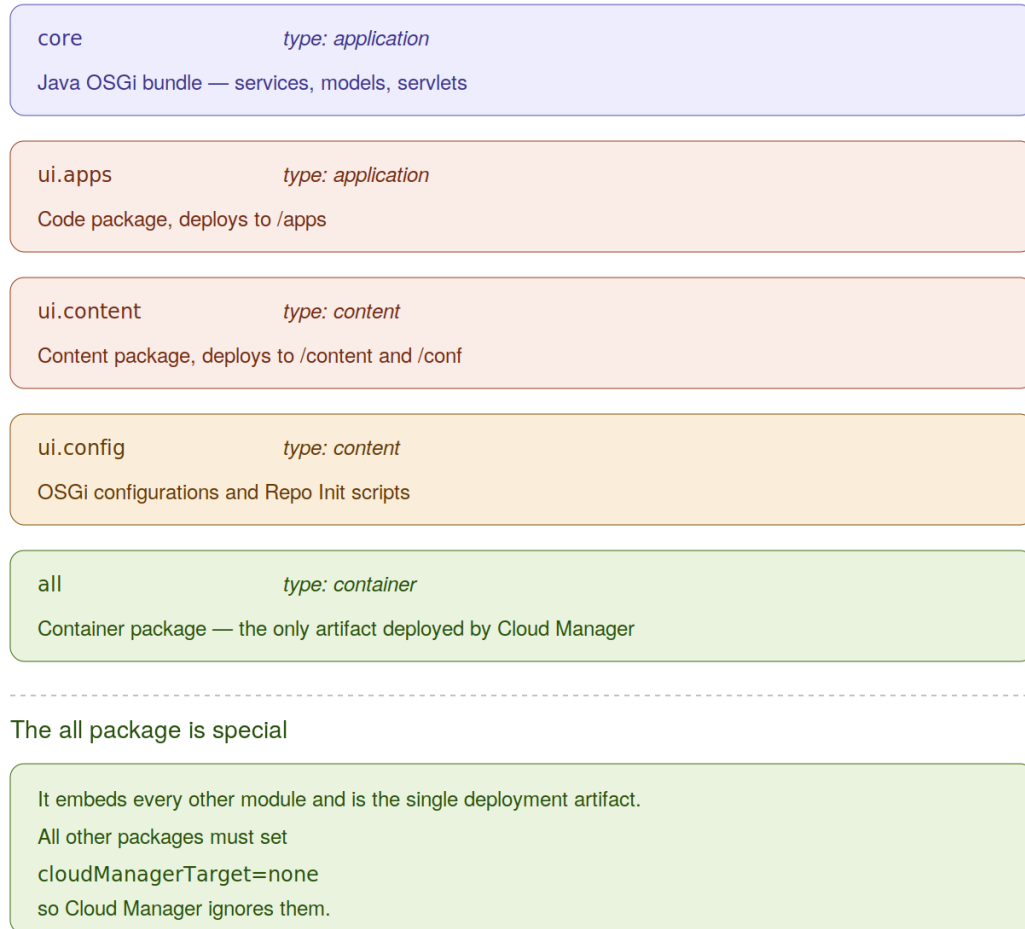


Figure 11. The Maven modules of a standard AEM project and their package types.

12. Repo Init and the AEM Archetype

Repo Init

Repo Init is a mechanism that defines JCR structures as part of the application.

You use it to create:

- Folder structures.
- Service users.
- Groups.
- ACL permissions.

These scripts run early in the deployment lifecycle, before any application code. They live in `ui.config` and are scoped by run mode.

AEM Project Archetype

To start a new project, you use the AEM Project Archetype. The current version is 56.

It generates the full project structure with best practices already in place:

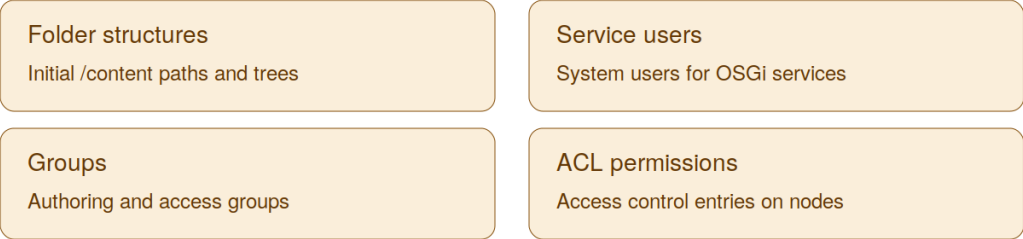
- `all`, `core`, `ui.apps`, `ui.content`, `ui.config`.
- `ui.frontend`, `tests`, and dispatcher configuration.

Repo Init and the AEM Project Archetype

Bootstrap repository structure and a new project

Repo Init

Defines JCR structures as part of the application.



Scripts run early in the deploy lifecycle — live in ui.config, scoped by run mode.

AEM Project Archetype (current version: 56)

Generates the full project structure with best practices in place.



The Archetype is the recommended starting point for any new AEM project.

It generates module wiring, parent POMs, run modes, and dispatcher templates automatically.

Figure 12. Repo Init capabilities and the modules generated by the AEM Archetype.

13. Troubleshooting

When something is not working, follow this sequence.

The five-step diagnostic flow

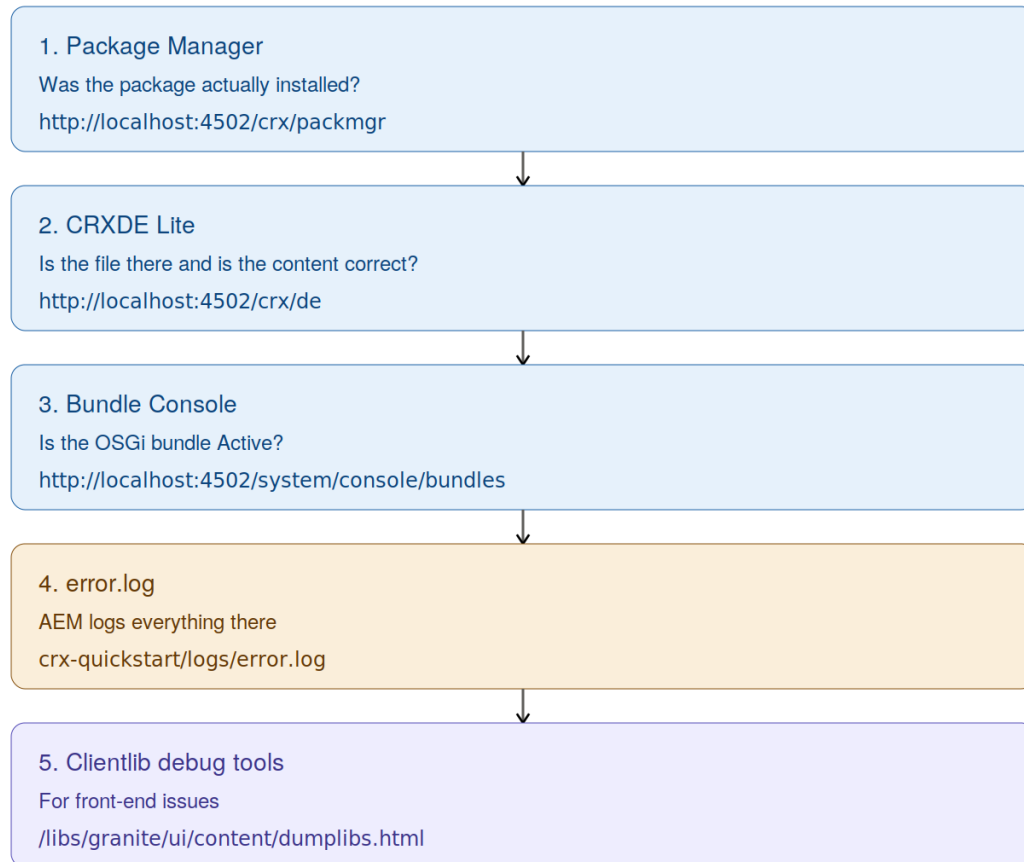
- 1. Check Package Manager — was the package actually installed?
<http://localhost:4502/crx/packmgr>
- 2. Check CRXDE Lite — is the file there and is the content correct?
<http://localhost:4502/crx/de>
- 3. Check the Bundle Console — is the OSGi bundle Active?
<http://localhost:4502/system/console/bundles>
- 4. Read the error.log — AEM logs everything there. Location: crx-quickstart/logs/error.log
- 5. For front-end issues — use the clientlib debug tools at
</libs/granite/ui/content/dumplib.html>

Most common errors

- Wrong Java version.
- Trying to double-click the QuickStart Jar.

Troubleshooting checklist

Run through these in order when something is not working



Most common errors

- Wrong Java version
- Trying to double-click the QuickStart Jar

Figure 13. The five-step troubleshooting checklist with URLs and log paths.

14. Closing

Setting up a local AEM environment is not just a setup task — it is the foundation for everything that follows.

What you now have

- The tools to build AEM projects.
- A local runtime to test your code.
- An understanding of how Maven packages are structured.
- A troubleshooting process when things go wrong.

Key concepts to remember for the exam

- QuickStart Jar naming convention and startup rules.
- Mutable vs immutable repository areas.
- The role of each Maven module (core, ui.apps, ui.content, ui.config).
- The all package as the single Cloud Manager deployment artifact.
- OAuth Server-to-Server is the only supported aio CLI authentication method (JWT removed January 2025).

Closing — what to remember

A working local environment is the foundation for everything else

Build tools

Java 21, Node, Maven, Git installed and verified

Local runtime

QuickStart Jar for Author and Publish

Local Dispatcher

Docker-based Apache with Dispatcher module

Project structure

Maven modules mapped to repository areas

Key concepts for the exam

- QuickStart Jar naming convention and startup rules
- Mutable vs immutable repository areas
- The role of each Maven module (core, ui.apps, ui.content, ui.config)
- The all package as the single Cloud Manager deployment artifact
- OAuth Server-to-Server is the only supported aio CLI auth (JWT removed Jan 2025)

Figure 14. Recap of the four pillars and the key concepts for the exam.